



University of Padua

Department of Information Engineering

Master Degree in Computer Engineering

Computer & Networks Security essay

NVIDIAScape: Container Escape via LD_PRELOAD Injection

May 2026

Candidate:

Alessio Demo

Student ID: 2142885

Professor:

Mauro Migliardi

Abstract

This essay analyses **NVIDIAScape** (CVE-2025-23266), a container-escape vulnerability affecting the NVIDIA Container Toolkit. The attack exploits the `LD_PRELOAD` environment variable, injected from inside a container, to force the privileged host-side hook (`nvidia-ctk`) to load attacker-controlled shared libraries. The root cause lies in the unsafe inheritance of the container environment by a process running with root privileges on the host, in violation of the principle of least privilege and proper isolation boundaries. The paper describes also the attack surface, the exploitation chain, the underlying design flaws, and the remediation measures introduced by NVIDIA and the broader container-security ecosystem.

Contents

1	Introduction	3
2	Background and Related Work	5
2.1	Linux Containers and Isolation Primitives	5
2.2	The NVIDIA Container Toolkit	5
2.3	The Dynamic Linker and LD_PRELOAD	6
2.4	Related Vulnerabilities	7
3	CVE-2025-23266 — Technical Analysis	9
3.1	Vulnerability Description	9
3.2	Root Cause: Unsafe Environment Inheritance	9
3.3	Attack Surface and Preconditions	10
3.4	Exploit Walkthrough	11
4	Exploitation Mechanics	13
4.1	Injecting LD_PRELOAD from the Container	13
4.2	Privileged Hook Execution on the Host	13
4.3	Escalation to Host Root	14
4.4	Realistic Attack Scenario: Cloud Multi-Tenant AI	14
5	Impact Assessment	16
5.1	CVSS Score and Breakdown	16
5.2	Impact on Kubernetes and Docker GPU Environments	16
5.3	Risk for LLM and AI Hosting Infrastructure	17
6	Mitigation and Hardening	19
6.1	NVIDIA’s Official Patch	19
6.2	Applying the Principle of Least Privilege	20
6.3	Environment Sanitisation at execve()	20
6.4	Defence-in-Depth	21
7	Broader Implications	23
7.1	Set-UID Architecture vs Service-Based Design	23
7.2	System Toolkits as Attack Vectors	23
7.3	The Future of GPU Security in AI Infrastructure	24
8	Conclusion	26
	References	27

1 Introduction

The rapid proliferation of containerized workloads has transformed the way computational resources are packaged, deployed, and shared across modern infrastructure. In particular we are talking about Containers, but what are they?

Containers are lightweight and portable execution environments built upon Linux kernel primitives such as namespaces, control groups, and security computer facilities, and in the last years have become the main deployment unit for cloud-native applications, data processing pipelines, artificial intelligence and machine learning workloads [1]. The emergence of large-scale AI training and inference has introduced a new and critical dimension to this landscape: the need to share hardware accelerators, most prominently **NVIDIA Graphics Processing Units (GPUs)**, across multiple isolated container workloads. Unlike CPU resources, which are abstracted transparently by the kernel scheduler, GPU access requires deep integration between the container runtime and the host operating system’s driver stack [2]. This integration is precisely where the attack surface examined in this paper resides. Going into details: The **NVIDIA Container Toolkit (nvidia-ctk)** is the software layer responsible for exposing GPU capabilities to containerised processes [3]. It acts as a privileged extension of the container runtime, executing hooks with root privileges on the host to configure the necessary device mappings and library bindings before the container process starts; the negative side of things is that it can be fatally exploited when the privileged process fails to sanitise the environment it inherits from an untrusted source. This vulnerability is identified as **CVE-2025-23266** [4] and allows a process running inside a container (potentially an unprivileged tenant in a shared cloud environment) to inject the LD_PRELOAD environment variable into the execution context of the host-side hook. Because the Linux dynamic linker processes LD_PRELOAD variable before any application code runs [5], the attacker’s malicious shared library is loaded and executed with full root privileges on the host, achieving a complete container escape. The significance of this vulnerability extends beyond its technical elegance. It strikes at a foundational assumption of container security: that isolation primitives reliably prevent a containerised process from affecting the host [6]. In the context of multi-tenant AI cloud services (where numerous organisations share the same physical GPU nodes) a single exploitable container can compromise the entire host, potentially exposing sensitive model weights, training data, and API keys.

This paper is structured as follows:

- Chapter 2 provides the necessary background on Linux container security, the NVIDIA Container Toolkit, the LD_PRELOAD mechanism, and historically related vulnerabilities.
- Chapter 3 presents a deep technical analysis of CVE-2025-23266 itself.
- Chapter 4 details the exploitation mechanics step by step.
- Chapter 5 assesses the real-world impact.
- Chapter 6 discusses mitigations and hardening strategies.
- Chapter 7 reflects on broader security implications.

2 Background and Related Work

2.1 Linux Containers and Isolation Primitives

Modern Linux containers do not rely on a distinct kernel or hardware boundary as virtual machines do. Instead, they exploit a layered set of kernel features to create the *illusion* of isolation while sharing the same underlying kernel with the host [1].

Namespaces. Linux namespaces partition global resources into per-process views [7]. The key namespaces for container isolation are: `pid` (process identifiers), `net` (network stack), `mnt` (filesystem mount points), `uts` (hostname and domain name), `ipc` (inter-process communication), and `user` (user and group identifiers). A process confined within these namespaces cannot observe or interact with resources belonging to other namespaces.

Control Groups (cgroups). Cgroups enforce resource accounting and limits — CPU time, memory, I/O bandwidth, and device access. They do not provide security isolation, but prevent resource exhaustion attacks and can restrict device access via the `devices` cgroup controller.

Seccomp. The Secure Computing Mode (`seccomp`) allows fine-grained filtering of system calls. Container runtimes like Docker apply a default seccomp profile that blocks hundreds of potentially dangerous syscalls from within the container.

Capabilities. Linux capabilities decompose the monolithic root privilege into discrete units (e.g., `CAP_NET_ADMIN`, `CAP_SYS_PTRACE`) [8]. Containers typically run with a reduced capability set, preventing common privilege escalation techniques that require specific capabilities. These primitives form a defence-in-depth stack [6], [9], however their security guarantees are only as strong as the correctness of the kernel implementation and the integrity of the privileged processes that configure and manage the container lifecycle. When a privileged process outside the container namespace interacts with untrusted input from within it (as in the case of NVIDIAScape) the isolation boundary can be subverted without any kernel vulnerability being required.

2.2 The NVIDIA Container Toolkit

The NVIDIA Container Toolkit is an open-source project that enables NVIDIA GPU access for containerised workloads [3]. It consists of several components:

- **libnvidia-container:** A shared library that performs the actual GPU device configuration.

- **nvidia-container-runtime**: A wrapper around standard OCI-compatible runtimes (e.g., **runc**) that injects NVIDIA-specific configuration.
- **nvidia-ctk**: The CLI tool and hook binary used to configure GPU access at container startup.

The toolkit integrates with the Open Container Initiative (OCI) specification through the **OCI hook** mechanism [10], [11]. OCI hooks are executable programs invoked by the container runtime at defined lifecycle points — **prestart**, **createRuntime**, **poststart**, and **poststop**. The NVIDIA **prestart** hook runs *on the host*, with root privileges, before the container process starts. Its role is to mount the necessary NVIDIA device files and libraries into the container’s filesystem [2].

Critically, the hook process is spawned in a context that inherits environmental information from the container’s configuration. As we shall examine in detail, this inheritance is the root cause of CVE-2025-23266.

The OCI hook specification (*runtime-spec*) does not mandate environment sanitisation for hook executables [10]. The security burden is implicitly placed on hook implementors, who must explicitly clean their execution environment when operating with elevated privileges.

2.3 The Dynamic Linker and LD_PRELOAD

The Linux dynamic linker (**ld.so** / **ld-linux.so**) is responsible for loading shared libraries required by an executable at runtime [5]. It resolves symbol dependencies, maps library segments into the process address space, and transfers control to the program’s entry point.

LD_PRELOAD. The **LD_PRELOAD** environment variable is a feature of the dynamic linker that allows specifying one or more shared libraries to be loaded *before all others* [5]. Its legitimate uses include debugging, profiling (e.g., replacing **malloc** with a tracking allocator), and binary instrumentation. The security implications of **LD_PRELOAD** are well-understood. The dynamic linker explicitly disables **LD_PRELOAD** (and related variables such as **LD_LIBRARY_PATH** and **LD_AUDIT**) when it detects that the effective user ID differs from the real user ID (the condition that characterises Set-UID programs [5]). This protection exists precisely because allowing untrusted **LD_PRELOAD** injection into a privileged process would enable trivial privilege escalation. However, this protection is only automatic for Set-UID binaries. For a process that *acquires* elevated privileges through other means (such as being spawned directly by a privileged parent) and does not explicitly sanitise its environment, **LD_PRELOAD** remains fully operative [12]. This is

the precise gap that NVIDIAScape exploits.

```

1 # For a SUID binary, the linker ignores LD_PRELOAD:
2 $ ls -la /usr/bin/sudo
3 -rwsr-xr-x 1 root root ... /usr/bin/sudo
4 $ LD_PRELOAD=/tmp/evil.so sudo whoami
5 # LD_PRELOAD is silently ignored (protection works)
6
7 # But for a root-spawned non-SUID hook process:
8 # The linker DOES honour LD_PRELOAD (no automatic protection)

```

Listing 1: The dynamic linker’s LD_PRELOAD security bypass for SUID binaries — not applicable when privilege is inherited from parent process.

2.4 Related Vulnerabilities

NVIDIAScape is not an isolated incident. It belongs to a broader family of container escape vulnerabilities that share a common pattern: a privileged host-side component incorrectly trusts or inherits input from within the container boundary [6].

CVE-2024-0132 — NVIDIA Container Toolkit (2024). A related vulnerability in an earlier version of the NVIDIA Container Toolkit allowed a crafted container image to mount sensitive host paths via time-of-check/time-of-use (TOCTOU) race conditions in the toolkit’s filesystem traversal logic [13], [14]. Unlike CVE-2025-23266, CVE-2024-0132 exploited filesystem handling rather than environment inheritance, but the underlying theme (insufficient trust boundary enforcement in a privileged GPU management layer) is identical.

Leaky Vessels (CVE-2024-21626 and related — runc, 2024). The “Leaky Vessels” cluster of vulnerabilities, disclosed in January 2024, affected the **runc** container runtime [15], [16]. CVE-2024-21626 allowed an attacker to leak a file descriptor referencing the host filesystem root by exploiting the order of operations in **runc**’s process initialisation. This enabled container escape with minimal preconditions. Much like NVIDIAScape, the vulnerability required no kernel exploit (only exploitation of a privileged userspace component’s incorrect handling of state that crossed the container boundary).

CVE-2019-5736 — runc. An earlier critical vulnerability in **runc** allowed a malicious container to overwrite the host **runc** binary by exploiting `/proc/self/exe` references during container execution [17]. The attacker could then execute arbitrary code as root the next time any container was started. This vulnerability established the now well-known principle that *container runtime processes themselves are attack surface*.

CVE-2022-0492 — cgroup v1. This kernel vulnerability allowed container escape via `cgroup` release agent abuse when the container had a weakened security profile [18]. While a kernel-level vulnerability rather than a userspace toolkit flaw, it reinforces the point that GPU and system management toolkits that run without full security profiles (no AppArmor, no seccomp) remove critical layers of defence.

These precedents collectively demonstrate that privileged host-side components managing container lifecycle (whether container runtimes, GPU toolkits, or storage drivers) represent a persistent and high-value attack surface that demands rigorous security engineering [1], [6].

3 CVE-2025-23266 — Technical Analysis

3.1 Vulnerability Description

CVE-2025-23266 is a **container escape vulnerability** in the NVIDIA Container Toolkit [4], [19]. It was publicly disclosed in 2025 and assigned a CVSS v3.1 base score of **9.0 (Critical)** [20], reflecting the combination of high confidentiality, integrity, and availability impact at the host level, with the attack vector being local (within the container) but resulting in complete host compromise.

The vulnerability affects versions of the NVIDIA Container Toolkit prior to the patched release and stems from a failure to sanitise the execution environment of the `nvidia-ctk` OCI hook process. Specifically, environment variables set within the container — including `LD_PRELOAD` — are inherited by the hook process when it is invoked by the container runtime on the host. Since the hook process runs with root privileges on the host, this inheritance allows an attacker inside the container to force the loading of an arbitrary shared library into the host’s privileged process space.

CVE-2025-23266 does not require any kernel vulnerability, any special container privileges (e.g., `-privileged`), or any network access. The only prerequisite is that the attacker can run a container on a host equipped with the NVIDIA Container Toolkit (a standard precondition in any GPU cloud environment).

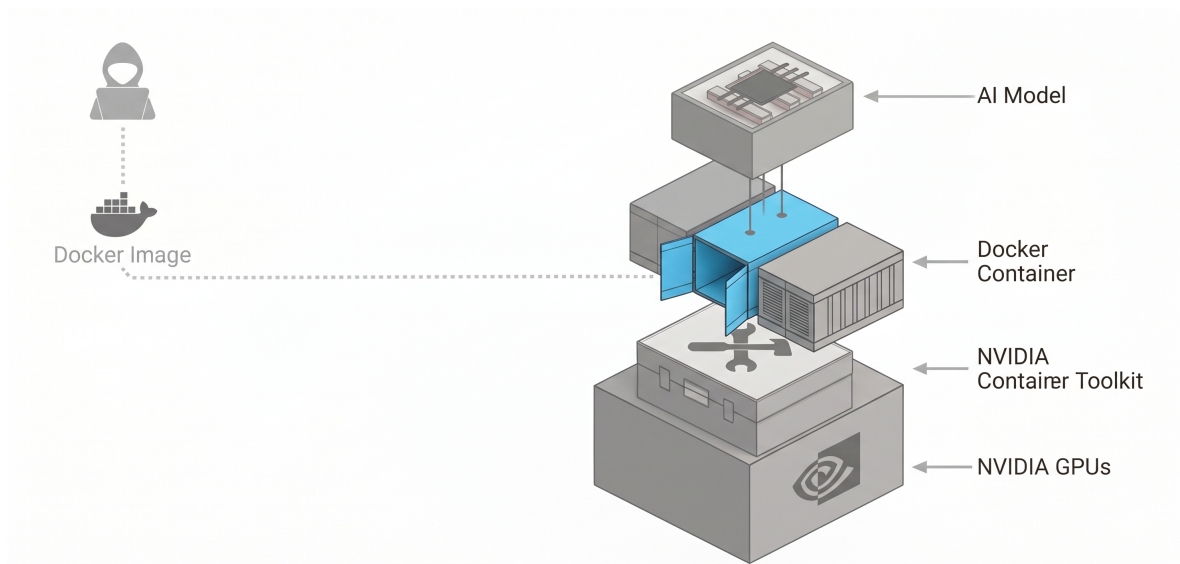


Figure 1: NVIDIA Container Toolkit architecture and hook execution flow.

3.2 Root Cause: Unsafe Environment Inheritance

To understand the root cause precisely, we must examine how the OCI hook is invoked. When a container with GPU access is started, the container runtime (e.g., `runc` or

containerd) reads the container’s OCI bundle specification (`config.json`), which includes a list of hooks. The NVIDIA prestart hook entry looks approximately as follows:

```
1 "hooks": {  
2   "prestart": [  
3     {  
4       "path": "/usr/bin/nvidia-ctk",  
5       "args": ["nvidia-ctk", "runtime", "hook", "prestart"],  
6       "env": []  
7     }  
8   ]  
9 }
```

Listing 2: Simplified OCI hook specification for nvidia-ctk (`config.json` excerpt).

The OCI runtime specification states that hook processes receive the container’s environment in the `env` field of the hook configuration. However, the problematic behaviour in the vulnerable versions of the toolkit arises from the fact that the hook process additionally inherits environment variables from its parent process (and the parent process’s environment is populated, directly or indirectly, from the container’s configured environment variables).

The crucial flaw is the absence of an explicit `execve()` call with a sanitised, minimal environment array before the hook begins its privileged operations [12]. In Unix process semantics, `execve()` is the mechanism by which a process can replace its image with a new executable; critically, it accepts an explicit `envp` array, allowing the caller to specify a completely controlled environment for the new process. A correctly written privileged process that must execute in an environment derived from untrusted input would call `execve()` with a hand-crafted, minimal `envp` containing only the variables strictly necessary for operation.

The vulnerable toolkit does not perform this sanitisation. The `LD_PRELOAD` variable, if present in the container’s environment, propagates to the hook process where the dynamic linker acts upon it faithfully [5] (loading the attacker-specified library with root privileges).

3.3 Attack Surface and Preconditions

The attack surface of CVE-2025-23266 is defined by the following preconditions:

1. **Vulnerable toolkit version:** The host must be running a version of the NVIDIA Container Toolkit prior to the patch. This affects a wide range of deployments that have not applied the security update.

2. **GPU-enabled container:** The attacker must be able to start a container with GPU access (e.g., using the `--gpus` flag in Docker or the `nvidia.com/gpu` resource request in Kubernetes). In shared cloud environments, this is a standard capability offered to paying customers.
3. **Ability to set environment variables:** The attacker must be able to define environment variables for their container. This is universally true for any container a user controls.
4. **Writable shared path:** The attacker's malicious shared library must be accessible to the host-side linker. This is achieved by placing the library in a path that is shared between the container and the host (such as a mounted volume or a device path accessible from both).

Notably absent from this list are: kernel exploits, elevated container capabilities, network access, and any form of social engineering. The attack is self-contained within the container startup sequence.

3.4 Exploit Walkthrough

The following provides a conceptual, step-by-step walkthrough of the exploit chain.

1. **Craft the malicious shared library.** The attacker writes a shared library containing a constructor function (a function annotated with `__attribute__((constructor))` in C) that executes the attacker's payload when the library is loaded. Because this runs before `main()`, it executes at the earliest possible moment in the host process lifecycle, before any security checks in the hook's application logic.

```
1 // evil.c -- compiled to evil.so
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 __attribute__((constructor))
6 void payload(void) {
7     // Executed with root privileges on the HOST
8     // at the moment the hook process loads this library.
9     // Example: spawn a reverse shell, write an SSH key,
10    // create a SUID shell binary, etc.
11    system("/bin/bash -c 'cp /bin/bash /tmp/rootshell"
12           " && chmod u+s /tmp/rootshell'");
13 }
```

Listing 3: Pseudocode for the malicious shared library constructor.

2. **Place the library on a shared path.** The library must be reachable from the host filesystem at the time the hook runs. The attacker mounts a host directory

into their container (a common practice for data I/O) and writes the compiled `evil.so` there. Alternatively, certain NVIDIA device paths mounted by the toolkit itself may be exploitable.

3. **Set LD_PRELOAD in the container environment.** When starting the container, the attacker sets the environment variable `LD_PRELOAD` to the host-side path of `evil.so`. In Docker CLI syntax:

```
1 docker run --gpus all \  
2   -e LD_PRELOAD=/mnt/shared/evil.so \  
3   -v /host/shared:/mnt/shared \  
4   attacker-image
```

Listing 4: Container launch with injected LD_PRELOAD.

4. **Hook invocation.** As part of the container startup sequence, the OCI runtime invokes the NVIDIA prestart hook (`nvidia-ctl runtime hook prestart`) on the host with root privileges. Due to the environment inheritance vulnerability, `LD_PRELOAD=/mnt/shared/evil.so` is present in the hook’s environment.
5. **Dynamic linker loads the malicious library.** The host’s dynamic linker reads `LD_PRELOAD` and maps `evil.so` into the hook process’s address space before any standard libraries. The constructor function executes immediately.
6. **Arbitrary code runs as root on the host.** The attacker’s payload runs with the full privileges of the hook process (root on the host) achieving a complete container escape. The container itself need not even finish starting for the escape to succeed.

The elegance of this exploit chain is its simplicity. No memory corruption, no kernel exploit, no race condition. A single environment variable and a compiled shared library are sufficient to escape the container and own the host.

4 Exploitation Mechanics

4.1 Injecting LD_PRELOAD from the Container

The mechanics of environment variable injection deserve careful examination. In the OCI container model, environment variables are defined in two places: the container image’s configuration (the `Env` field in the image manifest) [11] and the runtime configuration provided at container launch (via `docker run -e` or Kubernetes `env` fields in a Pod specification [21]).

Both mechanisms ultimately result in the environment variables being written into the container’s OCI `config.json` [10]. The vulnerable NVIDIA toolkit versions propagate this environment to the hook process through the parent process chain without filtering [19]. The following diagram illustrates the inheritance path:

```

1 Container Runtime (dockerd / containerd)
2   |
3   |-- reads config.json (contains LD_PRELOAD set by attacker)
4   |
5   v
6 runc (OCI runtime) -> forks hook process
7   |
8   |-- inherits environment from runtime process
9   |   (including LD_PRELOAD from container config)
10  |
11  v
12 nvidia-ctk hook [UID=0, running on HOST]
13   |
14   v
15 ld.so reads LD_PRELOAD -> loads evil.so -> constructor() executes

```

Listing 5: Environment propagation chain from container to privileged hook.

The key insight is that `runc` (or the equivalent runtime) spawns the hook process via `fork()` and `execve()`, but the `envp` argument passed to `execve()` (which should contain a clean, controlled environment) instead contains the attacker-influenced container environment.

4.2 Privileged Hook Execution on the Host

A critical aspect of the vulnerability is the privilege level at which the hook executes. OCI prestart hooks run in the host’s mount namespace and with the full privilege set of the container runtime daemon, which is typically root [10]. This is architecturally necessary: the hook must mount GPU device files into the container’s namespace (an operation that requires `CAP_SYS_ADMIN` and the ability to interact with the host’s device

filesystem) [8].

What makes this particularly dangerous is that the hook runs *before* the container’s security policies (seccomp profiles, AppArmor policies, dropped capabilities) have been applied to the container process. The hook itself is not subject to the container’s security configuration (it is a host process, not a container process [2]).

Furthermore, in most production deployments, the NVIDIA Container Toolkit’s hook binary is not subject to any additional Mandatory Access Control policy [19]. It runs as a plain root process with full Linux capabilities, making it an extremely high-value target. Once the attacker’s library is loaded into this process, there are effectively no further kernel-level restrictions preventing arbitrary host access.

4.3 Escalation to Host Root

The privilege escalation achieved through this vulnerability is immediate and unconditional. The constructor function in the malicious shared library executes in the security context of the `nvidia-ctk` hook process [4], which means:

- **UID 0 (root)** on the host system.
- **Full Linux capability set**, including `CAP_SYS_ADMIN`, `CAP_DAC_OVERRIDE`, `CAP_CHOWN`, and all others [8].
- **Host mount namespace access**, allowing reading and writing of any file on the host filesystem [7].
- **Host network namespace access** (depending on how the runtime is configured), potentially enabling network-level attacks against other hosts.

Practical payloads an attacker could deploy include: writing an SSH public key to `/root/.ssh/authorized_keys` for persistent access; creating a SUID copy of `/bin/bash` in a world-writable directory, modifying `/etc/passwd` or `/etc/sudoers` to add an unprivileged account with root access, installing a kernel rootkit or exfiltrating sensitive host secrets (API keys, TLS certificates, other container data) [14].

4.4 Realistic Attack Scenario: Cloud Multi-Tenant AI

To contextualise the real-world severity of NVIDIAScape, consider the following realistic attack scenario in a cloud AI infrastructure:

Setting. A cloud provider offers GPU-accelerated compute instances for AI model training and inference. Multiple customers share the same physical host, each running their workloads in isolated containers. The host runs the NVIDIA Container Toolkit (vulnerable version) to provide GPU access to all tenants.

Attacker. A malicious customer (Tenant A) provisions a standard GPU container instance (a service available to any paying customer with no special vetting). They have full control over their container’s environment variables and can mount volumes.

Attack sequence.

1. Tenant A writes a malicious shared library (`evil.so`) implementing a reverse shell or a persistent backdoor into a volume accessible from the host.
2. Tenant A starts their container with `LD_PRELOAD` pointing to `evil.so`.
3. The NVIDIA hook executes on the host, loads `evil.so`, and the payload runs as root.
4. The attacker now has root on the host. They can: read the memory, disk, and GPU state of all other tenants on the same host, exfiltrate proprietary model weights from Tenant B’s running container, extract API keys and credentials from environment variables of other containers, pivot to the cloud provider’s internal network via the host’s network interfaces.

Impact. The breach extends far beyond the attacker’s own container. All co-located tenants are compromised, the cloud provider’s infrastructure is at risk, and the attacker has achieved lateral movement potential into the provider’s broader internal network. [4], [14]. Remember that all these problems started from a standard, unprivileged GPU container.

5 Impact Assessment

5.1 CVSS Score and Breakdown

CVE-2025-23266 carries a **CVSS v3.1 Base Score of 9.0 (Critical)** [4], [20]. The breakdown of the vector string is as follows:

Table 1: CVSS v3.1 Vector Breakdown for CVE-2025-23266

Metric	Description	Value
Attack Vector (AV)	Attacker must have local access (inside a container)	Local (L)
Attack Complexity (AC)	No special conditions; attack is reliably repeatable	Low (L)
Privileges Required (PR)	Standard container user, no elevated privileges needed	Low (L)
User Interaction (UI)	No victim interaction required; purely automated	None (N)
Scope (S)	Impact escapes the vulnerable component (container)	Changed (C)
Confidentiality (C)	Complete host confidentiality loss	High (H)
Integrity (I)	Complete host integrity loss	High (H)
Availability (A)	Host availability can be fully disrupted	High (H)

The **(Metric = Scope, Value = Changed)** tuple is particularly noteworthy. It reflects the fact that the vulnerability’s impact transcends the security boundary of the initially vulnerable component (the container) and affects a separate, higher-privilege component (the host). This scope change is the defining characteristic of a container escape vulnerability and is the primary driver of the high base score.

5.2 Impact on Kubernetes and Docker GPU Environments

The vulnerability has significant implications across the two dominant GPU container deployment platforms.

Docker. In a standard Docker deployment with GPU support, the attack is straightforward as described in the exploit walkthrough (3.4). Any user with the ability to run GPU-enabled containers on a host with the vulnerable toolkit can exploit the vulnerability. In practice, Docker is widely used for development workstations, on-premises GPU servers, and small-scale cloud deployments [4].

Kubernetes. The impact in Kubernetes environments is potentially broader and more severe [21]. In a Kubernetes cluster with GPU node pools, the NVIDIA Container Toolkit is deployed as a DaemonSet on GPU nodes, making it universally present on all GPU-capable nodes [22]. An attacker who can submit a Pod specification with GPU resource requests and arbitrary environment variables (which is a standard capability for any authenticated Kubernetes user with access to a GPU namespace) can exploit the vulnerability on any GPU node the pod is scheduled on.

Moreover, Kubernetes introduces additional blast-radius concerns:

- The compromised node’s `kubelet` credentials can be extracted, potentially enabling lateral movement to the Kubernetes API server.
- The node’s service account tokens mounted into pods can be used to interact with the cluster’s API.
- In cloud-managed Kubernetes services (GKE, EKS, AKS), the node’s cloud instance metadata endpoint may be accessible, potentially yielding cloud IAM credentials with broad permissions.

5.3 Risk for LLM and AI Hosting Infrastructure

The intersection of NVIDIAScape with modern AI infrastructure deserves particular emphasis. The past several years have seen explosive growth in GPU-enabled container deployments driven by large language model (LLM) training and inference workloads. Organisations such as cloud providers, AI startups, and research institutions routinely operate dense GPU clusters where many tenants share physical nodes.

In this context, the threat model for NVIDIAScape is especially concerning for the following reasons:

High-value targets on GPU nodes. GPU nodes hosting LLM inference typically contain: proprietary model weights (often representing millions of dollars in training costs), API keys and secrets injected via environment variables into inference containers, customer data submitted for inference, and billing and identity information. A container escape on such a node yields immediate access to all of this.

Shared GPU nodes in inference clusters. Large-scale LLM serving infrastructure frequently uses bin-packing strategies that place multiple customer workloads on the same GPU node to maximise utilisation. This maximises the value of a single successful container escape.

Automated and autoscaling deployments. The automated nature of cloud-native GPU deployments means that vulnerable nodes may be continuously provisioned and deprovisioned without manual security review. Unless patch deployment is automated, vulnerable versions can persist in production for extended periods.

Regulatory and compliance implications. Organisations in regulated industries (healthcare, finance) that use GPU cloud infrastructure for AI workloads face additional exposure from NVIDIAScape: a successful exploit could result in cross-tenant data leakage that triggers data breach notification obligations under GDPR, HIPAA, or SOC 2 requirements.

6 Mitigation and Hardening

6.1 NVIDIA's Official Patch

NVIDIA released a patched version of the Container Toolkit that directly addresses the environment variable inheritance flaw. The fix introduces explicit environment sanitisation in the hook process startup path. Specifically, the patched code constructs a minimal, explicitly defined environment array before executing the hook's privileged operations, ensuring that variables such as LD_PRELOAD, LD_LIBRARY_PATH, LD_AUDIT, and other dynamic linker control variables are stripped from the hook's execution environment regardless of what the container configuration specifies.

Recommended action. All operators of GPU-enabled container infrastructure should immediately upgrade the NVIDIA Container Toolkit to the patched version. The upgrade process involves updating the `libnvidia-container`, `nvidia-container-toolkit`, and `nvidia-container-runtime` packages and restarting the container runtime daemon.

```
1 # Debian/Ubuntu
2 sudo apt-get update
3 sudo apt-get install -y nvidia-container-toolkit
4 sudo systemctl restart docker # or containerd
5
6 # RHEL/CentOS/Rocky Linux
7 sudo dnf update nvidia-container-toolkit
8 sudo systemctl restart docker
9
10 # Verify installed version
11 nvidia-ctl --version
```

Listing 6: Package upgrade commands for common Linux distributions.

In Kubernetes. The toolkit DaemonSet must be updated on all GPU nodes. Depending on the cluster configuration, this may require a node drain cycle to safely restart the container runtime without disrupting running GPU workloads.

CDI mode as architectural mitigation. As a complementary measure, operators should evaluate migrating to the Container Device Interface (CDI) mode supported by recent versions of the NVIDIA Container Toolkit [2]. CDI decouples device configuration from the OCI hook execution path: instead of a privileged hook running at container startup, device specifications are pre-generated on the host and consumed by the runtime as static configuration [23]. This removes the dynamic hook invocation entirely, eliminating the environment inheritance vector at an architectural level.

6.2 Applying the Principle of Least Privilege

Beyond patching the immediate vulnerability, the NVIDIAScape case highlights the importance of applying the **Principle of Least Privilege (PoLP)** to privileged system toolkit components [9].

The `nvidia-ctk` hook needs root access to perform specific, well-defined operations: mounting device files, configuring cgroups, and setting up library bind mounts. Rather than running as unrestricted root, a hardened implementation would:

- Use a minimal capability set, retaining only those capabilities strictly required (e.g., `CAP_SYS_ADMIN` for mount operations, `CAP_DAC_OVERRIDE` for device file access) and dropping all others explicitly via `cap_set_proc()`.
- Execute as a dedicated, non-login service account rather than the system root user, even when elevated capabilities are required.
- Implement privilege separation, performing privileged operations in a short-lived child process that immediately exits after completing its specific task, rather than maintaining elevated privileges throughout the hook’s lifecycle.

6.3 Environment Sanitisation at `execve()`

The technically correct fix at the code level (independent of the NVIDIA-specific patch) is the application of environment sanitisation at the `execve()` boundary. Any program that must execute with privileges derived from a context that includes untrusted environment variables should adopt the following pattern:

```

1 #include <unistd.h>
2
3 // Safe environment: only variables the hook legitimately needs
4 char *safe_env[] = {
5     "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
6     NULL // Terminator
7 };
8
9 // Re-execute self (or the actual hook binary) with clean environment
10 execve("/proc/self/exe", argv, safe_env);
11 // If execve returns, it failed -> handle the error
12 perror("execve");
13 exit(EXIT_FAILURE);

```

Listing 7: Correct environment sanitisation pattern using `execve()` with explicit envp.

This pattern is analogous to what Set-UID programs effectively benefit from automatically via the dynamic linker’s `AT_SECURE` flag. By explicitly constructing the

environment, the hook author removes any dependency on inherited state from untrusted sources.

An alternative approach is to use `clearenv()` to remove all environment variables and then re-add only those explicitly required. However, the `execve()` re-execution pattern is generally considered more robust, as it also eliminates any state that may have been inherited in process-internal structures (file descriptors, signal masks, etc.).

6.4 Defence-in-Depth

Patching the toolkit and sanitising execution environments address the specific vulnerability, but a robust security posture requires defence-in-depth [9](multiple independent layers of controls that limit the impact of any single failure).

Seccomp profiles. The `nvidia-ctk` hook process should be subject to a restrictive seccomp profile that allows only the system calls strictly necessary for its GPU configuration operations. At a minimum, this profile should block `execve()` with arbitrary arguments, preventing the hook from spawning attacker-controlled child processes. Docker and Kubernetes support custom seccomp profiles that can be applied to specific components [21], [24].

AppArmor / SELinux policies. Mandatory Access Control frameworks can confine the hook process to a specific, minimal set of filesystem paths and operations. An AppArmor profile for `nvidia-ctk` would, for example, allow writes only to specific `/dev/nvidia*` paths and the container's rootfs device mount points, while denying writes to sensitive host paths such as `/root`, `/etc`, and `/home` [25].

Rootless containers. Running the container runtime in rootless mode (where the container runtime itself operates as an unprivileged user) significantly reduces the blast radius of hook-based attacks. In rootless mode, the hook process does not have true root privileges on the host, and its ability to affect host-wide state is constrained. While GPU support for rootless containers has historically been limited, recent versions of the NVIDIA toolkit have improved rootless GPU support.

Read-only and no-new-privileges flags. Containers should be started with the `no-new-privileges` security option whenever possible. While this does not directly prevent NVIDIAScape (the exploit occurs in the hook, not the container process), it limits the post-escape options available to an attacker if they attempt to escalate further from within the container.

Network segmentation and egress filtering. Even if an attacker achieves host root, network-level controls can limit the damage. Strict egress filtering on GPU nodes (allowing only expected outbound connections) can prevent reverse shells and data exfiltration attempts from succeeding, buying time for incident response.

7 Broader Implications

7.1 Set-UID Architecture vs Service-Based Design

The NVIDIAScape vulnerability illuminates a fundamental architectural tension in the design of privileged system software. Two broad paradigms exist for components that must perform privileged operations on behalf of unprivileged callers:

Set-UID / Direct Execution Model. In this model, a binary is given elevated privileges via the Set-UID bit or capabilities, and is invoked directly by the unprivileged caller. The binary runs in an environment largely inherited from the caller. Security depends entirely on the binary’s own defences (proper environment sanitisation, capability dropping, input validation, and so forth). The dynamic linker provides one automatic protection (`AT_SECURE`) for SUID binaries, but this does not cover all inheritance paths. Historical experience shows that SUID programs are a persistent source of privilege escalation vulnerabilities.

Service / IPC Model. In this model, the privileged operations are performed by a long-running daemon (a service) that runs as root or with specific capabilities, and that receives requests from unprivileged clients via a well-defined inter-process communication (IPC) channel (Unix socket, D-Bus, etc.). This model is architecturally more secure because the trust boundary is explicit: the service validates each incoming request independently of the caller’s environment.

The `nvidia-ctk` hook architecture is closer to the Set-UID model: it is invoked directly, inheriting environmental state from the runtime chain. Had the GPU toolkit been designed as a privileged service (e.g., a Unix socket daemon receiving structured hook invocation requests), the `LD_PRELOAD` injection vector would not exist.

This architectural lesson is broadly applicable: system-level toolkits that bridge unprivileged workloads and privileged hardware or OS resources should prefer service-based designs over direct invocation models wherever feasible.

7.2 System Toolkits as Attack Vectors

CVE-2025-23266 is a reminder that **the software infrastructure around containers is as much a part of the attack surface as the containers themselves**. The security community’s focus on container hardening is valuable but incomplete if the privileged components of the container ecosystem are not equally rigorous.

Hardware management layers (GPU drivers, storage drivers, network plugins, device plugins in Kubernetes) represent a particularly attractive attack surface for several

reasons:

- They are maintained by hardware vendors whose primary expertise is hardware enablement, not security engineering.
- They require deep integration with the operating system, often necessitating elevated privileges that the container runtime itself does not require.
- They are frequently treated as trusted infrastructure, exempt from the security scrutiny applied to application containers.
- Their update cadence may lag behind security disclosures, particularly in regulated or conservative enterprises.

The practical implication is that security teams responsible for container infrastructure must extend their threat models to include the privileged toolkit layer. Vulnerability scanning tools should explicitly track versions of container toolkits and plugins alongside the container runtime and kernel versions. Security-sensitive deployments should have explicit policies for toolkit update response times following CVE disclosure.

7.3 The Future of GPU Security in AI Infrastructure

The trajectory of AI infrastructure development suggests that the attack surface represented by NVIDIAScape will grow rather than shrink in the coming years.

Increasing GPU density and multi-tenancy. As GPU compute becomes more expensive and in demand, infrastructure providers will continue to maximise hardware utilisation through dense multi-tenancy. This amplifies the impact of any single container escape, making individual vulnerabilities more consequential.

MIG and GPU partitioning. NVIDIA’s Multi-Instance GPU (MIG) technology allows a single physical GPU to be partitioned into multiple isolated instances for simultaneous use by different workloads [26]. While MIG provides hardware-level isolation between GPU workloads, the Container Toolkit still manages the host-side configuration of MIG instances (meaning that vulnerabilities in the toolkit remain relevant even in MIG deployments [2], [26]).

Confidential computing for AI. The emerging field of confidential computing aims to provide cryptographic isolation of workloads even from the host operator [27]. NVIDIA’s Confidential Computing mode encrypts GPU memory and attestation state, making it inaccessible to the host. However, the Container Toolkit must still interface with the host to configure confidential GPU instances, and vulnerabilities in this configuration path remain a potential concern [2].

Supply chain security. As AI infrastructure becomes critical infrastructure, the security of the supply chain for GPU management software will attract increasing attention from both defenders and attackers. Code signing, reproducible builds, and formal security reviews for toolkit releases will become more important [3].

Regulatory pressure. Emerging AI regulation [28] increasingly mandates security controls for AI systems that process sensitive data. This will drive demand for formal security assessments of GPU infrastructure components, including container toolkits, and may accelerate the adoption of secure-by-design architectural patterns.

In summary, NVIDIAScape is not an anomaly but a leading indicator of a broader challenge: securing the privileged software infrastructure that connects AI workloads to the physical hardware they depend on. Meeting this challenge requires architectural improvements in toolkit design, rigorous vulnerability management programs, and a security posture that treats the container toolchain as a first-class threat surface.

8 Conclusion

This paper has presented a comprehensive analysis of NVIDIAScape (CVE-2025-23266), a critical container escape vulnerability arising from unsafe environment variable inheritance in the NVIDIA Container Toolkit’s privileged OCI hook. We have examined the technical root cause (the absence of environment sanitisation at the `execve()` boundary) and traced the precise exploit chain by which an attacker within an unprivileged container can achieve arbitrary code execution with root privileges on the host.

The vulnerability exemplifies a class of security failures that recurs across the history of privileged system software: the unsafe consumption of untrusted input inherited through the process execution chain. In this specific case, the inherited input is the `LD_PRELOAD` environment variable, and the dynamic linker’s faithful processing of it becomes the mechanism of exploitation. The exploit requires no kernel vulnerability, no elevated container permissions, and no user interaction, only a standard GPU container and control over its environment variables.

The impact in modern AI infrastructure is severe. GPU nodes hosting multi-tenant LLM workloads represent some of the most data-rich and commercially sensitive targets in current cloud infrastructure, and the ubiquity of the NVIDIA Container Toolkit across GPU deployments means that the vulnerability had a very broad blast radius at the time of disclosure.

Several lessons emerge from this analysis. First, privileged system toolkits must treat their execution environment as untrusted when that environment is derived, directly or indirectly, from unprivileged input. Second, the Principle of Least Privilege must be applied not only to container workloads but equally to the privileged components of the container ecosystem itself. Third, service-based architectural designs for hardware management layers are inherently more robust than direct invocation models that inherit caller state. Fourth, defence-in-depth controls (Mandatory Access Control policies, seccomp profiles, rootless container runtimes) provide critical residual protection when individual components fail.

As AI workloads continue to demand GPU resources at scale, and as the infrastructure supporting them grows in complexity and strategic value, the security of the privileged software layer connecting containers to hardware will remain a critical and underexplored frontier. NVIDIAScape is a precise and instructive case study in both the fragility of this layer and the clear engineering principles that can make it more robust.

References

- [1] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” *IEEE Access*, vol. 7, pp. 52 976–52 996, 2019. DOI: [10.1109/ACCESS.2019.2911732](https://doi.org/10.1109/ACCESS.2019.2911732). [Online]. Available: <https://ieeexplore.ieee.org/document/8693491>.
- [2] NVIDIA Corporation, *NVIDIA Container Toolkit Documentation*, NVIDIA Developer Docs, Accessed: 2025, 2025. [Online]. Available: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/index.html>.
- [3] NVIDIA Corporation, *nvidia-container-toolkit — GitHub Repository*, GitHub, Accessed: 2025, 2025. [Online]. Available: <https://github.com/NVIDIA/nvidia-container-toolkit>.
- [4] National Vulnerability Database, *CVE-2025-23266 Detail*, NIST NVD, Accessed: 2025, 2025. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2025-23266>.
- [5] Linux man-pages project, *ld.so(8) — Linux Programmer’s Manual*, Linux Manual Pages, Accessed: 2025, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man8/ld.so.8.html>.
- [6] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, “A measurement study on linux container security: Attacks and countermeasures,” in *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC)*, 2018, pp. 418–429. DOI: [10.1145/3274694.3274720](https://doi.org/10.1145/3274694.3274720). [Online]. Available: <https://dl.acm.org/doi/10.1145/3274694.3274720>.
- [7] Linux man-pages project, *namespaces(7) — Linux Programmer’s Manual*, Linux Manual Pages, Accessed: 2025, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
- [8] Linux man-pages project, *capabilities(7) — Linux Programmer’s Manual*, Linux Manual Pages, Accessed: 2025, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/capabilities.7.html>.
- [9] M. Souppaya, J. Morello, and K. Scarfone, “Application Container Security Guide (NIST SP 800-190),” National Institute of Standards and Technology, Special Publication 800-190, 2017, Accessed: 2025. DOI: [10.6028/NIST.SP.800-190](https://doi.org/10.6028/NIST.SP.800-190). [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>.
- [10] Open Container Initiative, *OCI Runtime Specification — Hooks*, Open Container Initiative, Accessed: 2025, 2024. [Online]. Available: <https://github.com/opencontainers/runtime-spec/blob/main/config.md#posix-platform-hooks>.

- [11] Open Container Initiative, *OCI Image Format Specification*, Open Container Initiative, Accessed: 2025, 2024. [Online]. Available: <https://github.com/opencontainers/image-spec>.
- [12] Linux man-pages project, *execve(2) — Linux Programmer's Manual*, Linux Manual Pages, Accessed: 2025, 2024. [Online]. Available: <https://man7.org/linux/man-pages/man2/execve.2.html>.
- [13] National Vulnerability Database, *CVE-2024-0132 Detail — NVIDIA Container Toolkit*, NIST NVD, Accessed: 2025, 2024. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2024-0132>.
- [14] Wiz Research, *NVIDIA AI Vulnerability Deep Dive: CVE-2024-0132 Container Escape*, Wiz Research Blog, Accessed: 2025, 2024. [Online]. Available: <https://www.wiz.io/blog/nvidia-ai-vulnerability-deep-dive-cve-2024-0132>.
- [15] National Vulnerability Database, *CVE-2024-21626 Detail — runc Container Escape (Leaky Vessels)*, NIST NVD, Accessed: 2025, 2024. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2024-21626>.
- [16] Snyk Security Research, *Leaky Vessels: Container Escapes in runc and Build-Kit (CVE-2024-21626)*, Snyk Blog, Accessed: 2025, 2024. [Online]. Available: <https://snyk.io/blog/leaky-vessels-docker-runc-container-breakout-vulnerabilities/>.
- [17] National Vulnerability Database, *CVE-2019-5736 Detail — runc Host Binary Overwrite*, NIST NVD, Accessed: 2025, 2019. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>.
- [18] National Vulnerability Database, *CVE-2022-0492 Detail — Linux Kernel cgroup v1 Container Escape*, NIST NVD, Accessed: 2025, 2022. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-0492>.
- [19] NVIDIA Corporation, *NVIDIA Container Toolkit Security Bulletin — CVE-2025-23266*, NVIDIA Product Security, Accessed: 2025, 2025. [Online]. Available: <https://www.nvidia.com/en-us/security/>.
- [20] FIRST.org, *Common Vulnerability Scoring System v3.1 Specification Document*, FIRST, Accessed: 2025, 2019. [Online]. Available: <https://www.first.org/cvss/specification-document>.
- [21] The Kubernetes Authors, *Kubernetes Security Documentation — Pod Security Standards*, Kubernetes.io, Accessed: 2025, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.
- [22] The Kubernetes Authors, *Kubernetes Device Plugin Framework Documentation*, Kubernetes.io, Accessed: 2025, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/>.

- [23] CNCF Tags / Container Device Interface, *Container Device Interface (CDI) Specification*, GitHub — cncf-tags/container-device-interface, Accessed: 2025, 2024. [Online]. Available: <https://github.com/cncf-tags/container-device-interface>.
- [24] Docker Inc., *Seccomp Security Profiles for Docker*, Docker Documentation, Accessed: 2025, 2024. [Online]. Available: <https://docs.docker.com/engine/security/seccomp/>.
- [25] Canonical Ltd., *AppArmor* — *Ubuntu Server Documentation*, Ubuntu Documentation, Accessed: 2025, 2024. [Online]. Available: <https://documentation.ubuntu.com/server/how-to/security/apparmor/>.
- [26] NVIDIA Corporation, *Multi-Instance GPU (MIG) User Guide*, NVIDIA Developer Docs, Accessed: 2025, 2024. [Online]. Available: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html>.
- [27] NVIDIA Corporation, *NVIDIA Confidential Computing — Hopper Architecture*, NVIDIA Developer Blog, Accessed: 2025, 2024. [Online]. Available: <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>.
- [28] European Parliament and Council, *Regulation (EU) 2024/1689 — Artificial Intelligence Act*, Official Journal of the European Union, Accessed: 2025, 2024. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>.